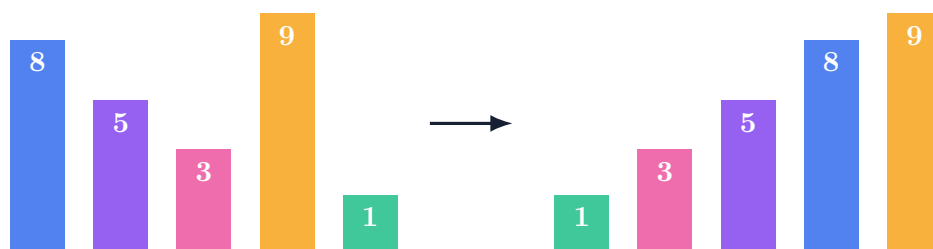


Bubble Sort

The Simplest Way to Understand Sorting Algorithms



Beginner friendly



Python included



Big ideas made visual

A premium visual handbook for learning your first sorting algorithm

June 11, 2026

Contents

1	What Is Bubble Sort?	3
2	Why Do We Need Sorting?	3
3	How Bubble Sort Works	4
3.1	Pass 1	4
3.2	Pass 2	4
3.3	Pass 3	5
4	Bubble Sort Animation Style Walkthrough	5
5	Bubble Sort Algorithm	5
5.1	Plain English Algorithm	5
5.2	Pseudocode	6
5.3	Flowchart	6
6	Python Implementation	7
6.1	Basic Version	7
6.2	Optimized Version	7
7	Real-Life Scenarios	8
8	When Bubble Sort Is Used	8
9	When Bubble Sort Should Not Be Used	9
10	Time Complexity	9
11	Space Complexity	9
12	Advantages and Disadvantages	10
13	Comparison with Other Sorting Algorithms	10
14	Five Fully Worked Examples	10
14.1	Example 1: Very Easy	10
14.2	Example 2: Easy	10
14.3	Example 3: Basic	11
14.4	Example 4: Medium	11
14.5	Example 5: Hard	11
15	Common Mistakes	11
16	Interview Questions	12

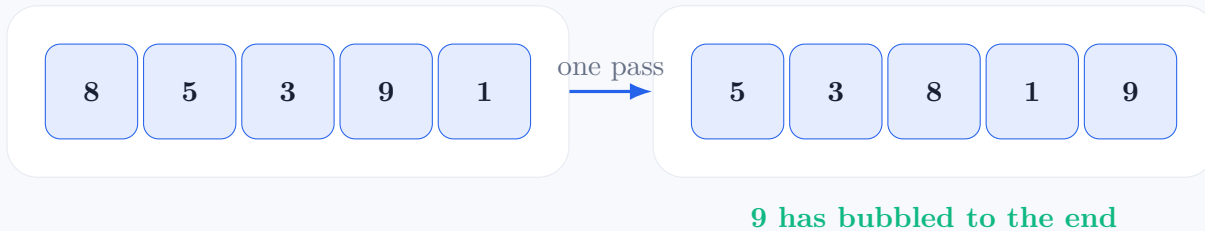
17 Practice Questions	12
18 Practice Question Solutions	12
19 Summary Cheat Sheet	13
20 Key Takeaways	13

1 What Is Bubble Sort?

Simple definition

Bubble Sort is a sorting method that repeatedly compares two neighboring items. If they are in the wrong order, it swaps them. After enough passes, the list becomes sorted.

Imagine bubbles in a glass of soda. Small, light bubbles rise toward the top. In Bubble Sort, the biggest number slowly “bubbles” to the end of the list during each pass.



😊 Fun fact

The name is visual: large values float rightward one step at a time, just like bubbles moving through water.

2 Why Do We Need Sorting?

Sorting turns messy information into useful information. A sorted list is easier to search, read, compare, and understand.

🔍 Searching

A dictionary is useful because words are alphabetized. Sorted data helps us find things faster.

🏆 Ranking systems

Games, contests, and leaderboards use sorting to show winners.

📁 Organizing data

Files sorted by name or date are easier to browse.

🗄️ Databases

Apps sort prices, dates, names, and ratings all the time.

★ Key takeaway

Sorting is like cleaning a room: once everything has a place, it becomes easier to find what you need.

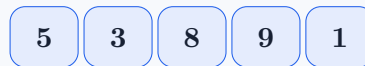
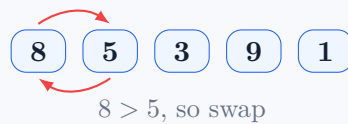
3 How Bubble Sort Works

We will sort this list from smallest to largest:

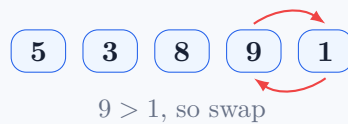


3.1 Pass 1

Compare neighbors from left to right.



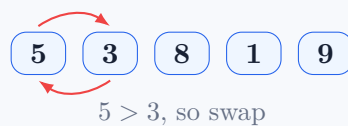
Compare 8 and 9: already okay.



💡 Tip

After Pass 1, the largest number, 9, is locked at the end.

3.2 Pass 2



Compare 5 and 8: already okay.



3 5 1 8 9

3.3 Pass 3

Compare 3 and 5: already okay.

3 5 1 8 9
5 > 1, so swap

3 1 5 8 9

More passes continue until no wrong neighbors remain:

1 3 5 8 9

4 Bubble Sort Animation Style Walkthrough

Before 8 5 3 9 1

Swap 8 and 5 5 8 3 9 1

Swap 8 and 3 5 3 8 9 1

Keep 8 and 9 5 3 8 9 1

Swap 9 and 1 5 3 8 1 9

★ Key takeaway

Bubble Sort is just a repeated question: “Are these two neighbors in the right order?”

5 Bubble Sort Algorithm

5.1 Plain English Algorithm

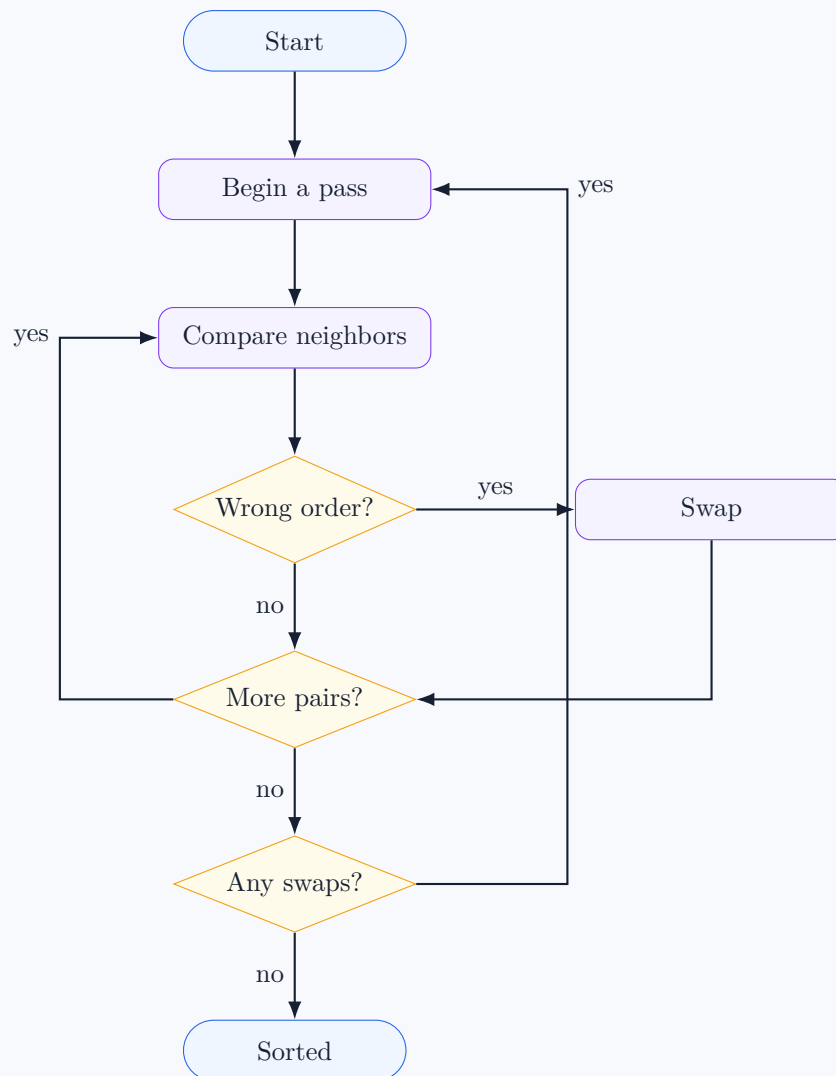
1. Start at the beginning of the list.
2. Compare the first two numbers.

3. If the left number is bigger, swap them.
4. Move one step to the right and compare the next pair.
5. Continue until the end of the list.
6. Repeat passes until the list is sorted.

5.2 Pseudocode

```
for each pass from 1 to n - 1:  
  for each pair of neighbors not already sorted:  
    if left item is bigger than right item:  
      swap them
```

5.3 Flowchart



6 Python Implementation

6.1 Basic Version

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n - 1):
5         for j in range(n - 1 - i):
6             if arr[j] > arr[j + 1]:
7                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8
9     return arr
```

Line-by-line explanation

- `n = len(arr)` counts how many items are in the list.
- The outer loop controls how many passes we make.
- The inner loop compares neighboring pairs.
- `if arr[j] > arr[j + 1]` checks whether two neighbors are backward.
- The swap line trades their positions.

6.2 Optimized Version

```
1 def bubble_sort_optimized(arr):
2     n = len(arr)
3
4     for i in range(n - 1):
5         swapped = False
6
7         for j in range(n - 1 - i):
8             if arr[j] > arr[j + 1]:
9                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10                swapped = True
11
12        if not swapped:
13            break
14
15    return arr
```

 Tip

If a full pass makes no swaps, the list is already sorted. The optimized version stops early instead of wasting time.

7

Real-Life Scenarios

Scenario	Why Bubble Sort can be used	Is it a good choice?
Student marks	It can arrange a small class from lowest to highest score.	Good for teaching; slow for a whole school.
Books by size	Compare two books at a time and swap them.	Fine for a small shelf.
Files by name	Names can be compared alphabetically.	Better algorithms are used in real file managers.
Products by price	Prices can be ordered from cheap to expensive.	Okay for a tiny demo shop.
Game scores	Scores can be ranked after a small game.	Good for a few players.
Educational software	The steps are visible and easy to animate.	Excellent for learning.
Small datasets	Few items mean speed is not a big problem.	Reasonable.
Learning algorithms	Shows loops, comparisons, and swaps clearly.	One of the best first examples.

8

When Bubble Sort Is Used

Small data

Ten items? Bubble Sort is easy enough.

Simple demos

Great for animations and whiteboard lessons.

Teaching

Students can see every comparison and swap.

Nearly sorted lists

With early stopping, it can finish quickly.

9 When Bubble Sort Should Not Be Used

⚠ Watch out

Bubble Sort is usually not the right tool for large or important sorting jobs.

Avoid it for:

- Large databases with thousands or millions of records.
- Real-time apps where every millisecond matters.
- Search engines, maps, banking systems, and streaming platforms.
- Production software that already has a built-in efficient sort.

10 Time Complexity

Time complexity asks: “How much work does the algorithm do as the list grows?”

Case	Meaning	Complexity	Friendly explanation
Best	Already sorted with optimization	$O(n)$	One quick check pass.
Average	Random order	$O(n^2)$	Many comparisons and swaps.
Worst	Reverse order	$O(n^2)$	Almost every pair must move.

What does $O(n^2)$ mean?

If the list gets longer, the work grows much faster. Doubling the list can create about four times as much work.

11 Space Complexity

Bubble Sort sorts the list in place. It only needs a tiny amount of extra memory for loop counters and swapping.

Original list changes inside the same memory box:

1

3

5

8

9

★ Key takeaway

Space complexity is $O(1)$ because Bubble Sort uses constant extra space.

12 Advantages and Disadvantages

👍 Advantages

- Very easy to understand.
- Simple to code.
- Great for teaching swaps.
- Uses little extra memory.
- Can stop early if optimized.

👎 Disadvantages

- Slow for large lists.
- Makes many comparisons.
- Makes many swaps.
- Rarely used in serious production sorting.
- Better algorithms exist.

13 Comparison with Other Sorting Algorithms

Algorithm	Speed	Complexity	Memory	Ease
Bubble Sort	Slow	$O(n^2)$	$O(1)$	Easiest to visualize.
Selection Sort	Slow	$O(n^2)$	$O(1)$	Finds the smallest each pass.
Insertion Sort	Good for nearly sorted data	$O(n^2)$	$O(1)$	Like sorting playing cards.
Merge Sort	Fast	$O(n \log n)$	$O(n)$	Splits and merges.
Quick Sort	Very fast on average	$O(n \log n)$ average	$O(\log n)$ typical	Uses pivots.

14 Five Fully Worked Examples

14.1 Example 1: Very Easy

Input: 2 1

- Pass 1: compare 2 and 1, swap → 1 2
- Output: 1 2

14.2 Example 2: Easy

Input: 3 1 2

- Pass 1: $3 > 1$ swap → 1 3 2; $3 > 2$ swap → 1 2 3.

- Pass 2: $1 < 2$ no swap.

• Output: **1** **2** **3**

14.3 Example 3: Basic

Input: **4** **2** **1** **3**

- Pass 1: $4\ 2\ 1\ 3 \rightarrow 2\ 4\ 1\ 3 \rightarrow 2\ 1\ 4\ 3 \rightarrow 2\ 1\ 3\ 4$.
- Pass 2: $2\ 1\ 3\ 4 \rightarrow 1\ 2\ 3\ 4$.
- Pass 3: no swaps.

• Output: **1** **2** **3** **4**

14.4 Example 4: Medium

Input: **5** **1** **4** **2** **8**

- Pass 1: $5\ 1\ 4\ 2\ 8 \rightarrow 1\ 5\ 4\ 2\ 8 \rightarrow 1\ 4\ 5\ 2\ 8 \rightarrow 1\ 4\ 2\ 5\ 8$.
- Pass 2: $1\ 4\ 2\ 5\ 8 \rightarrow 1\ 2\ 4\ 5\ 8$.
- Pass 3: no swaps.

• Output: **1** **2** **4** **5** **8**

14.5 Example 5: Hard

Input: **9** **7** **5** **3** **1**

- Pass 1: $9\ 7\ 5\ 3\ 1 \rightarrow 7\ 9\ 5\ 3\ 1 \rightarrow 7\ 5\ 9\ 3\ 1 \rightarrow 7\ 5\ 3\ 9\ 1 \rightarrow 7\ 5\ 3\ 1\ 9$.
- Pass 2: $7\ 5\ 3\ 1\ 9 \rightarrow 5\ 7\ 3\ 1\ 9 \rightarrow 5\ 3\ 7\ 1\ 9 \rightarrow 5\ 3\ 1\ 7\ 9$.
- Pass 3: $5\ 3\ 1\ 7\ 9 \rightarrow 3\ 5\ 1\ 7\ 9 \rightarrow 3\ 1\ 5\ 7\ 9$.
- Pass 4: $3\ 1\ 5\ 7\ 9 \rightarrow 1\ 3\ 5\ 7\ 9$.

• Output: **1** **3** **5** **7** **9**

15

Common Mistakes

⚠ Watch out

Most Bubble Sort bugs come from loop boundaries.

- Using `range(n)` inside the inner loop and accidentally checking `arr[j + 1]` past the end.
- Forgetting to swap both values.
- Forgetting that the last `i` items are already sorted.
- Thinking one pass always sorts the whole list.
- Adding early stopping but never setting `swapped = True`.

16

Interview Questions

1. **What is Bubble Sort?** A sorting algorithm that compares neighbors and swaps them if they are in the wrong order.
2. **Why is it called Bubble Sort?** Large values bubble toward the end of the list during each pass.
3. **What is the worst-case time complexity?** $O(n^2)$.
4. **What is the best case with optimization?** $O(n)$ when the list is already sorted.
5. **Is Bubble Sort stable?** Yes. Equal items keep their original relative order if we only swap when the left item is greater.

17

Practice Questions

Do these first. The next section contains solutions.

1. **Very Easy:** Sort `[2, 1]` using Bubble Sort.
2. **Easy:** Show all passes for `[3, 2, 1]`.
3. **Basic:** Count the swaps needed for `[4, 1, 3, 2]`.
4. **Medium:** Write optimized Bubble Sort in Python.
5. **Hard:** Explain why reverse-sorted input is the worst case.

18

Practice Question Solutions

1. **Very Easy:** Compare 2 and 1. Swap. Result: `[1, 2]`.
2. **Easy:** Pass 1: `[3,2,1] -> [2,3,1] -> [2,1,3]`. Pass 2: `[2,1,3] -> [1,2,3]`.
3. **Basic:** `[4,1,3,2] -> [1,4,3,2]` swap 1; `-> [1,3,4,2]` swap 2; `-> [1,3,2,4]` swap 3; `-> [1,2,3,4]` swap 4. Total: 4 swaps.

4. **Medium:** Use a **swapped** flag. If a pass makes no swaps, break early.
5. **Hard:** In reverse order, almost every neighboring pair is wrong. Each item must move many steps, causing the maximum comparisons and swaps.

19

Summary Cheat Sheet

 Bubble Sort at a Glance**Definition**

Compare neighbors. Swap if wrong. Repeat.

Key idea

Big values bubble right; small values drift left.

Complexity

Best: $O(n)$ optimized. Average/Worst: $O(n^2)$.

Space: $O(1)$.

Use cases

Teaching, small lists, simple animations, nearly sorted tiny data.

Advantages

Easy, visual, in-place, stable.

Disadvantages

Slow for large lists.

```
1 def bubble_sort(arr):
2     for i in range(len(arr) - 1):
3         for j in range(len(arr) - 1 - i):
4             if arr[j] > arr[j + 1]:
5                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
6     return arr
```

20

Key Takeaways

★ Key takeaway

- Bubble Sort compares neighboring items and swaps them when needed.
- It is one of the easiest sorting algorithms to learn and draw.
- It is useful for teaching, small examples, and visual demonstrations.
- It should not be used for large or performance-critical datasets.
- Its average and worst-case time complexity is $O(n^2)$.
- Its space complexity is $O(1)$ because it sorts in place.